SPRINT 2

Nivel 1

Ejercicio 1

A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen.Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.

A continuación, pongo las instrucciones para crear la base de datos “transactions” desde el documento ESTRUCTURA DE DATOS y las ejecuto una por una.

Nuestra base de datos TRANSACTIONS consta de dos tablas Company y Transaction
Aquí podemos ver las columnas de la tabla de Company:

```
CREATE TABLE IF NOT EXISTS company (
    id VARCHAR(15) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(15),
    email VARCHAR(100),
    country VARCHAR(100),
    website VARCHAR(255)
);
```

La tabla Company tiene 6 columnas y su PK es la columna “id” que a la vez se relaciona con la segunda tabla Transaction a través de su columna “company_id” que es FK. La relación es de uno a muchos. Cada empresa solo tiene un id en la tabla de Company, pero su id puede salir muchas veces en la tabla de transacciones. Su Primary Key es la columna “id” (de las transacciones). Esta tabla tiene 8 columnas y 100 registros.

```
SELECT *
 FROM transactions.company;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings

company 1 ×

Action Output

#	Time	Action	Message	Duration / Fetch
1	23:17:04	SELECT * FROM transactions.company	100 row(s) returned	0.000 sec / 0.000 sec

Aquí podemos ver todas las columnas de la tabla de transacciones. Esta tabla tiene 100000 registros.

```
CREATE TABLE IF NOT EXISTS transaction (
    id VARCHAR(255) PRIMARY KEY,
    credit_card_id VARCHAR(15) REFERENCES credit_card(id),
    company_id VARCHAR(20),
    user_id INT REFERENCES user(id),
    lat FLOAT,
    longitude FLOAT,
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,
    FOREIGN KEY (company_id) REFERENCES company(id)
);
```

```
SELECT * FROM transactions.transaction;
```

Action Output

#	Time	Action	Message	Duration / Fetch
1	23:26:28	SELECT * FROM transactions.transaction	100000 row(s) returned	0.000 sec / 0.219 sec

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

Refresh data re-executing the original query

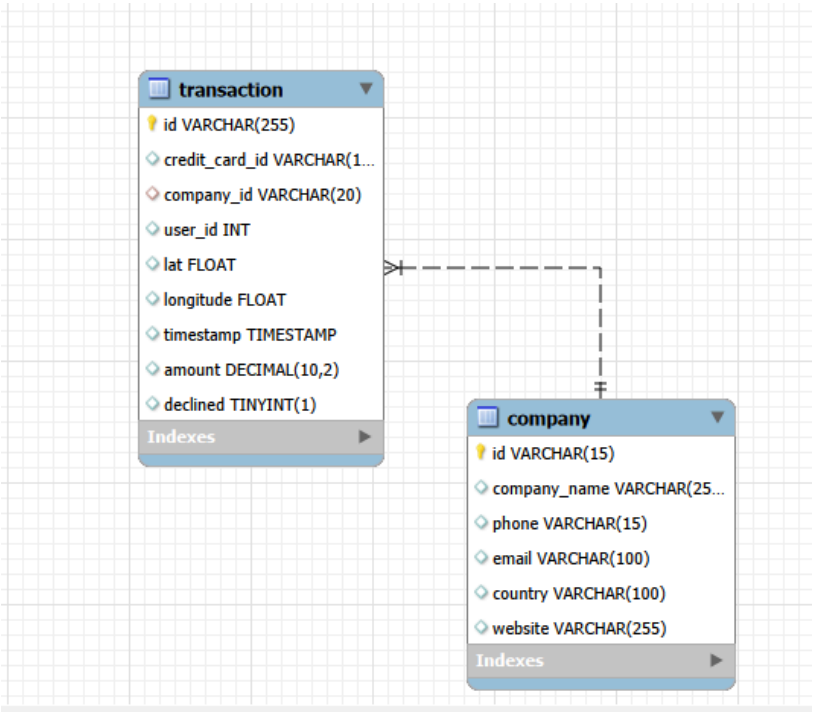
id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-4[illegible]	[illegible]	2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
000447FE-B650-4[illegible]	[illegible]	2370	438	41.5972	12.2218	2016-12-21 20:07:18	155.63	0
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0

Ahora rellenamos nuestras tablas al ejecutar el código del segundo documento sql "datos_introducir".

```
USE transactions;

-- Insertamos datos de company
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (       'b-2222', 'Ac Fermentum Incorporated', '06 85 56 52 33', 'donec.p
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (       'b-2226', 'Magna A Neque Industries', '04 14 44 64 62', 'risus.do
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (       'b-2230', 'Fusce Corp.', '08 14 97 58 85', 'risus@protonmail.edu'
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (       'b-2234', 'Convallis In Incorporated', '06 66 57 29 50', 'mauris.
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (       'b-2238', 'Ante Iaculis Nec Foundation', '08 23 04 99 53', 'sed.d
```

Para acceder al diagrama de la base de datos, voy al menú superior, pulso Database → Reverse Engineer.



Ejercicio 2

Utilizando JOIN realizarás las siguientes consultas: Listado de los países que generan ventas. A través de JOIN hago la consulta uniendo transaction y company, filtrando fuera transacciones que han sido rechazadas (no llegó a crearse VENTA).

```
48 •  SELECT DISTINCT c.country FROM company c
49    JOIN transaction t
50    ON c.id = t.company_id
51    WHERE t.declined <>1 -- porque las transacciones fallidas no las debemos incluir
52    ORDER BY c.country ASC;
53
54
55
56     -- Identifica a la compañía con la mayor media de ventas
57 •  SELECT c.company_name, ROUND(AVG(t.amount),2) as media
58    FROM company c
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country
NULL
Australia
Belgium
Canada
China

Result 4 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:02:33	SELECT DISTINCT c.country FROM company c JOIN transaction t ON c.id = t.company_id ...	16 row(s) returned	0.234 sec / 0.000 sec

Desde cuántos países se generan las ventas. Hay 15 países desde los cuales se generan las ventas. Identifica a la compañía con la mayor media de ventas.

Agrupo por el nombre de la empresa y ordeno por la media de las cantidades de ventas (amount).

```
57 •  SELECT c.company_name, ROUND(AVG(t.amount),2) as media
58    FROM company c
59    JOIN transaction t
60    ON c.id=t.company_id
61    WHERE t.declined <>1
62    GROUP BY c.company_name
63    ORDER BY media DESC
64    LIMIT 1;
65
66
67    -- Para practicar window function opcion dos
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	media
Ac Fermentum Incorporated	284.91

Result 5 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:03:34	SELECT c.company_name, ROUND(AVG(t.amount),2) as media FROM company c JOIN tran...	1 row(s) returned	0.719 sec / 0.000 sec

EJERCICIO 3

Utilizando sólo subconsultas (sin utilizar JOIN).Muestra todas las transacciones realizadas por empresas de Alemania.

Usando la subconsulta con “IN” busca las coincidencias en ID en tabla transacions y company y con el filtro en la tabla de company, por pais– Alemania.

```
79 •  SELECT * FROM transaction
80    WHERE company_id IN (
81    SELECT id
82    FROM company
83    WHERE country ="Germany"
84    );
85      -- Lista las empresas que han realizado transacciones por un monto superior a la media de todas las transacciones.
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

transaction 97 | Apply

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:37:19	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company WHERE ...	13291 row(s) returned	0.000 sec / 0.062 sec

Lista las empresas que han realizado transacciones por un monto superior a la media de todas las transacciones.

```
74 •  SELECT company_name
75    FROM company
76    WHERE id IN (
77    SELECT company_id
78    FROM transaction
79    WHERE amount > (
80    SELECT AVG(amount) FROM transaction)
81    );
82
83
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Iaculis Nec Foundation

company 8 | Read Only

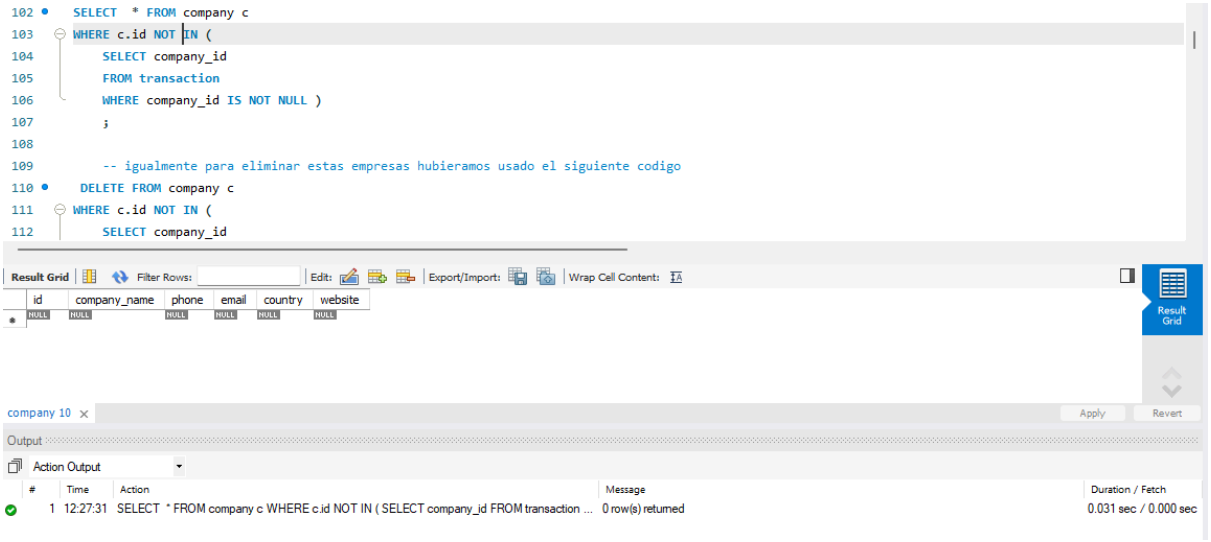
Output

Action Output

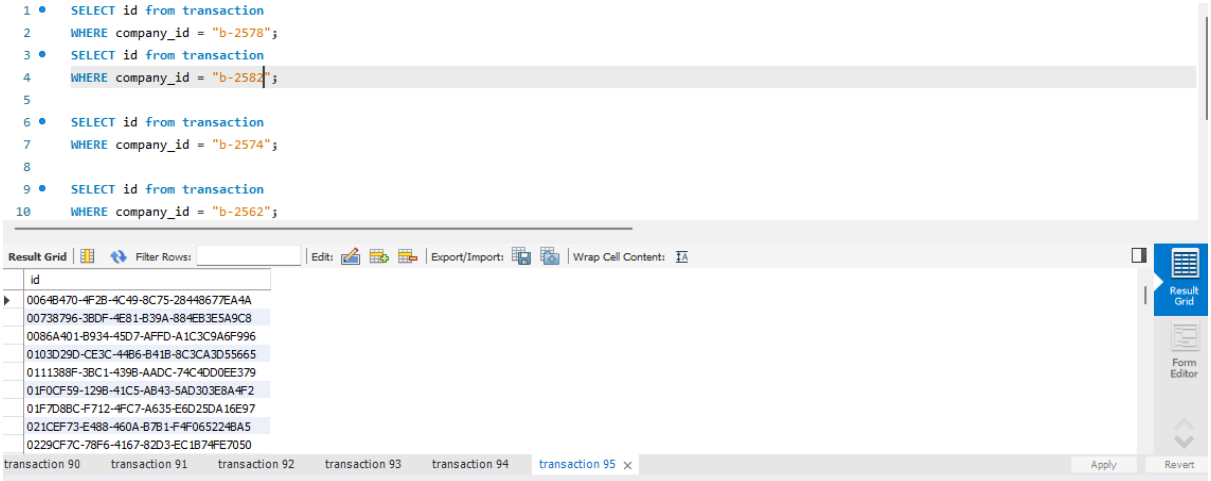
#	Time	Action	Message	Duration / Fetch
1	23:57:47	SELECT company_name FROM company WH...	100 row(s) returned	0.078 sec / 0.000 sec

Elimina del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

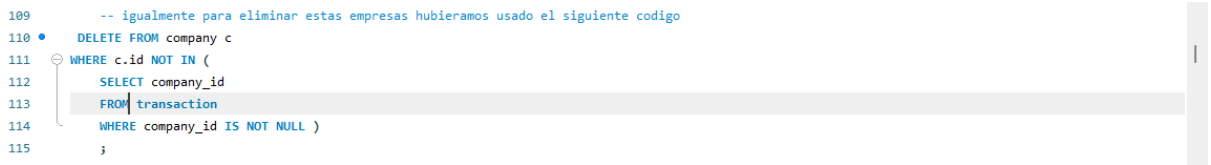
Primero hago una consulta para sacar la lista de empresas que no tienen transacciones registradas y, como podemos observar, no existen en esta base de datos empresas que no hayan realizado ninguna transacción.



Como no estaba segura de mí misma, lo comprobé de forma manual aunque sé que es una pérdida de tiempo.



Igualmente, para eliminar los registros que coinciden con la última consulta usaremos el siguiente código:



Ejercicio 4

Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de

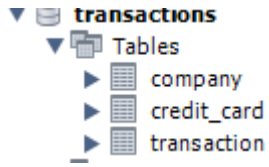
crear la tabla, será necesario que ingreses la información del documento denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

Para saber qué columnas tengo que crear, miro el documento "datos_introducit_credit_card".

```
117 • CREATE TABLE IF NOT EXISTS credit_card (
118   id VARCHAR(15),
119   iban VARCHAR(34),
120   pan VARCHAR(19),
121   pin VARCHAR(4),
122   cvv VARCHAR(3),
123   expiring_date VARCHAR(10)
124   );
125   -- Procedemos a llenar nueva tabla con los registros del documento numero 4 "datos_introducir_credit_card" y despues reviso como se v
```

```
Output
Action Output
#  Time      Action                                                                  Message              Duration / Fetch
1  11:46:38  CREATE TABLE IF NOT EXISTS credit_card ( id VARCHAR(15), iban VARCHAR(34), pan VA...  0 row(s) affected  0.032 sec
```

Comprobamos que la tabla existe al actualizar el esquema.



Procedemos a llenar la nueva tabla con los registros del documento "datos_introducit_credit_card". Comprobamos que tiene 5000 registros.

```
1   -- Insertamos datos de credit_card
2 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR330950312213576817638661', '5424465566813633', '3257', '984', '1
3 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'DO28054763748537475216568689', '5142423821948828', '9080', '887',
4 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 453 55 5287', '4598', '438', '06/29
5 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583', '667', '02/24/2
6 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566 886747 7265', '4900', '130', '10/
7 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT8780622813509242945634 6', '544 58654 54343 384', '8760', '887',
8 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2980', 'DE39241881883086277136', '402400 7145845969', '5075', '596', '07/2
9 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2987', 'GE89681434837748781813', '3763 747687 76666', '2298', '797', '10/3
10 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', 'BH6271442836806676529 4', '344283273252593', '7545', '595', '02/28/
11 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CY490874266547745812668321 10', '511722 924833 2244', '9562', '867'
12 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU507216693616119230', '4485744464433884', '1856', '740', '04/05/2
13 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS119398216295715968342456821', '3784 662233 17389', '3246', '822'
```

```
#     Time      Action                                                                  Message              Duration / Fetch
4999  11:54:58  INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX78...  1 row(s) affected  0.000 sec
5000  11:54:58  INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX91...  1 row(s) affected  0.000 sec
```

Luego reviso como se han cargado los datos pero con un LIMIT 10

```sql
SELECT * FROM credit_card
LIMIT 10;

-- Modificamos tipos de atributos en la nueva tabla credit card
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	10/30/22
CcU-2945	DO26854763748537475216568689	5142423821948828	9080	887	08/24/23
CcU-2952	BG45IVQL52710525608255	4556 453 55 5287	4598	438	06/29/21
CcU-2959	CR7242477244335841535	372461377349375	3583	667	02/24/23
CcU-2966	BG72LKTQ15627628377363	448566 886747 7265	4900	130	10/29/24

credit_card 99 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5000	11:54:58	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX9...	1 row(s) affected	0.000 sec
5001	11:55:44	SELECT * FROM credit_card LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

Procedemos a revisar los tipos de datos y a limpiar la tabla credit_card.
Miramos si hay duplicados:

```sql
--- limpieza de la tabla. revisamos si hay duplicados
SELECT id, COUNT(*)
FROM credit_card
GROUP BY id
HAVING COUNT(id) <> 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	COUNT(*)

Result 100 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5002	11:58:35	SELECT COUNT(id) FROM credit_card WHERE COUNT(id) <> 1	Error Code: 1111. Invalid use of group function	0.000 sec
5003	11:59:18	SELECT id, COUNT(*) FROM credit_card GROUP BY id HAVING COUNT(id) <> 1	0 row(s) returned	0.031 sec / 0.000 sec

Miramos si hay NULL:

```sql
SELECT *
FROM credit_card
WHERE id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	iban	pan	pin	cvv	expiring_date

credit_card 101 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:00:36	SELECT * FROM credit_card WHERE id IS NULL	0 row(s) returned	0.015 sec / 0.000 sec

AHORA PASAMOS A REVISAR TIPOS DE COLUMNAS:
Primero cambiaré el formato y despues el tipo de la columna expiring_date a DATE y asigno PK al id. No me lo deja hacer sin SQL_SAFE_UPDATES

```sql
SET SQL_SAFE_UPDATEs =0;
UPDATE credit_card
SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')
WHERE id IS NOT NULL;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:12:53	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y') WHER...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.328 sec

```sql
ALTER TABLE credit_card
MODIFY COLUMN expiring_date DATE,
ADD PRIMARY KEY (id);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:14:06	ALTER TABLE credit_card MODIFY COLUMN expiring_date DATE, ADD PRIMARY KEY (id)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.266 sec

Cierro el SQL_SAFE_UPDATES

```sql
SET SQL_SAFE_UPDATEs =1;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:14:06	ALTER TABLE credit_card MODIFY COLUMN expiring_date DATE, ADD PRIMARY KEY (id)	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.266 sec
2	12:15:47	SET SQL_SAFE_UPDATEs =1	0 row(s) affected	0.000 sec

Ahora, reviso si los cambios se ejecutaron correctamente con DESCRIBE

```sql
-- vuelvo a revisar como quedo la tabla credit_card
DESCRIBE credit_card;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(34)	YES		NULL	
pan	varchar(19)	YES		NULL	
pin	varchar(4)	YES		NULL	
cvv	varchar(3)	YES		NULL	
expiring_date	date	YES		NULL	

Result 104 | Read Only

Finalmente, tenemos que crear una relación entre la nueva tabla credit_card y transaction, en la tabla transaction tenemos columna "credit_card_id"
que será la FK relacionada a "id" de la tabla credit_card que es su PK*/

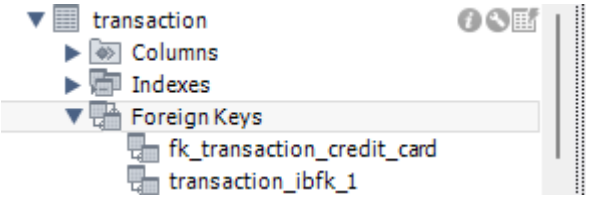
```sql
/*ahora tenemos que crear relacion entre la nueva tabla credit_card y transaction, en la tabla transaction tenemos columna "credit_card_id"
que sera la FK relacionada a "id" de la tabla credit_card que es su PK*/

ALTER TABLE transaction
ADD CONSTRAINT fk_transaction_credit_card
FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:24:15	SELECT * FROM credit_card	5000 row(s) returned	0.000 sec / 0.016 sec
2	11:35:22	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_credit_card FOREIGN KEY (cr...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.219 sec



Comprobamos que la FK se ha creado con éxito.

Arquitectura del Modelo (Diseño en Estrella)
La base de datos presenta una arquitectura con un modelo en estrella.

En este diseño, la tabla transaction actúa como la tabla de hechos (Fact Table) central, ya que registra los eventos y movimientos operativos del negocio. A su vez, las tablas company y credit_card funcionan como tablas de dimensiones (Dimension Tables).

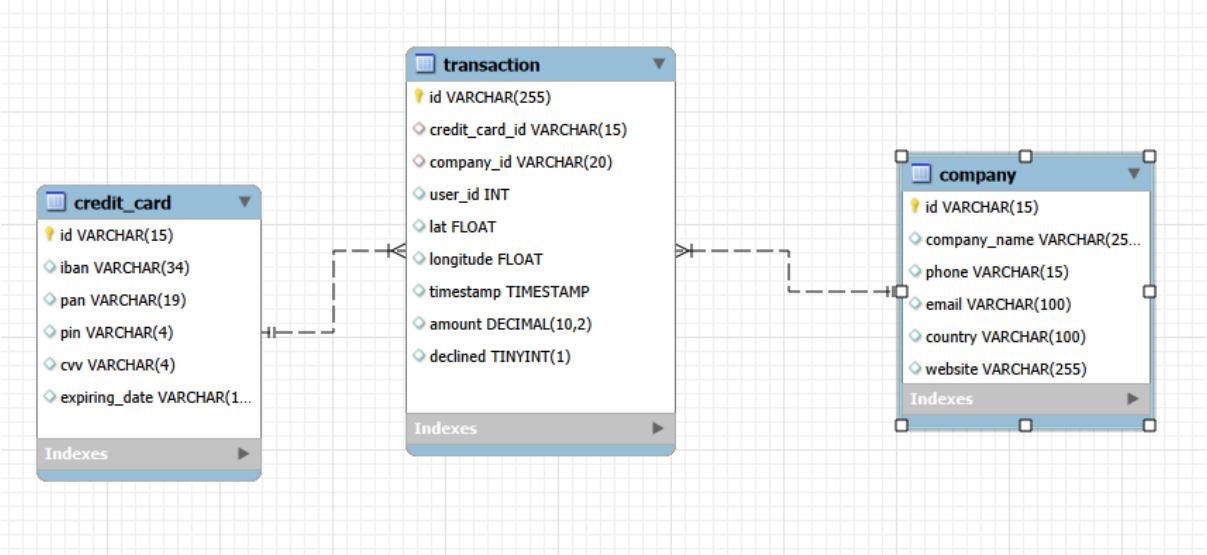
2.Relación company ➔ transaction es de uno a muchos / 1:N

Una empresa (company) puede realizar muchas transacciones a lo largo del tiempo. Sin embargo, cada transacción individual pertenece de forma exclusiva a una sola empresa. La relación se establece mediante la clave foránea company_id en la tabla transaction, la cual hace referencia a la clave primaria id de la tabla company.

3. Relación transaction ➔ credit_card es de muchos a uno / N:1.

Puede haber muchas transacciones registradas con la misma tarjeta de crédito. Sin embargo, solo puede existir un id para cada tarjeta.

Se realiza a través de la clave foránea credit_card_id en la tabla transaction, enlazada con la clave primaria id de la tabla credit_card.



Ejercicio 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es:

TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

```
-- Ejercicio 5
/*El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938.
 La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.*/
--  Modifico el número de cuenta (IBAN) de la tarjeta
UPDATE credit_card
SET iban = 'TR323456312213576817699999'
WHERE id = 'CcU-2938';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:46:17	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = '...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec

Aquí comprobamos que el cambio se realizó con éxito.

```
-- Reviso que esta correctamente
SELECT id, iban
FROM credit_card
WHERE id = 'CcU-2938';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	iban
CcU-2938	TR323456312213576817699999
NULL	NULL

credit_card 10 | Apply | Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:47:21	SELECT id, iban FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec

EJERCICIO 6 En la tabla "transaction" ingresa una nueva transacción. Cuando lo intento hacer, me encuentro con el problema que la empresa no existe en la tabla "company" y el número de tarjeta no existe en la tabla "credit_card".

```
INSERT INTO transaction (id, credit_card_id,company_id,user_id,lat, longitude, amount, declined)
VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD","CcU-9999","b-9999","9999","829.999","-117.999","111.11", "0");
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:47:21	SELECT id, iban FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec
2	11:51:24	INSERT INTO transaction (id, credit_card_id,company_id,user_id,lat, longitude, a...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (...	0.000 sec

Así que primero las tengo que crear en dichas tablas, ya que las tablas están relacionadas.

```
142 •  INSERT INTO company (id) VALUES ("b-9999");
143
144
145
146
147
148
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:55:07	INSERT INTO company (id) VALUES ("b-9999")	1 row(s) affected	0.015 sec

```
188 •  INSERT INTO credit_card (id) VALUES ("CcU-9999");
189
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:33:09	INSERT INTO credit_card (id) VALUES ("CcU-9999")	1 row(s) affected	0.000 sec

Ahora sí podemos crear la nueva transacción:

```
145     -- ahora podemos anadir nuevo registro en transaction
146 •  INSERT INTO transaction (id, credit_card_id,company_id,user_id,lat, longitude, amount, declined)
147     VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD","CcU-9999","b-9999","9999","829.999","-117.999","111.11", "0");
148
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:56:48	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ("CcU-9...	1 row(s) affected	0.016 sec
2	11:57:19	INSERT INTO transaction (id, credit_card_id,company_id,user_id,lat, longitude, am...	1 row(s) affected	0.000 sec

Finalmente revisamos si se ha creado con éxito el nuevo registro:

```
149     -- Comprobamos si se guardo de forma correcta
150 •  SELECT * FROM transactions.transaction
151     WHERE id = "108B1D1SD-5B23-A76C-55EF-C568E49A99DD";
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	NULL	111.11	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor

transaction 11 | Apply | Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:58:43	SELECT * FROM transactions.transaction WHERE id = "108B1D1D-5B23-A76C-...	1 row(s) returned	0.000 sec / 0.000 sec

EJERCICIO 7

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

Primero eliminamos la columna.

```
156 •  ALTER TABLE credit_card
157    DROP COLUMN pan;
158
159
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:01:31	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec

Segundo, revisamos como queda la tabla con DESCRIBE:

```
160 •  DESCRIBE credit_card;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(34)	YES		NULL	
pin	varchar(4)	YES		NULL	
cvv	varchar(4)	YES		NULL	
expiring_date	varchar(10)	YES		NULL	

Result Grid | Form Editor

Result 12 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:02:03	DESCRIBE credit_card	5 row(s) returned	0.000 sec / 0.000 sec

EJERCICIO 8 CREAMOS NUEVA BASE DE DATOS que yo llamé “transact”

```
162    -- Ejercicio 8 Crear una base de datos nuevo con datos de varios CSVs.alter
163 •  CREATE DATABASE IF NOT EXISTS transact;
164
165
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:02:03	DESCRIBE credit_card	5 row(s) returned	0.000 sec / 0.000 sec
2	12:24:27	CREATE DATABASE IF NOT EXISTS transact	1 row(s) affected	0.000 sec

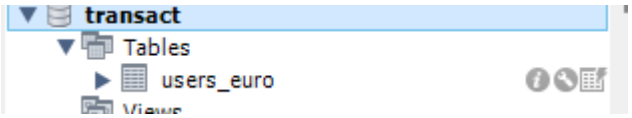
A continuación, creamos las tablas: transactions, credit_cards, european_users, american_users y companies.
Empezamos por crear las tablas de dimensiones y terminaremos con la tabla de hechos, transactions ya que de lo contrario nos daría el error al describir las foreign keys de tablas que no existiesen.
Primero creo la tabla de usuarios europeos, pero ya con la idea de unificarla con la tabla de usuarios de EE. UU. Para ello, ya creo una columna nueva llamada región.

```sql
CREATE TABLE users_euro (
    id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(30),
    email VARCHAR(150),
    birth_date VARCHAR(50),
    country VARCHAR(100),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address VARCHAR(255),
    signup_date VARCHAR(50),
    user_segment VARCHAR(50),
    income_band VARCHAR(50),
    region VARCHAR(10)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	14:08:36	CREATE TABLE users_euro (id VARCHAR(50) PRIMARY KEY, name VARC...	0 row(s) affected	0.031 sec



Pasamos a cargar los datos de esta tabla con el comando de LOAD DATA.

```sql
LOAD DATA
INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__european_users.csv" -- anadí doble diagonal al ver el video de V.Hu
INTO TABLE users_euro
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, name, surname, phone, email, birth_date, country, city,
 postal_code, address, signup_date, user_segment, income_band)
SET region = 'EU'
;

CREATE TABLE IF NOT EXISTS transactions(
id VARCHAR(50) PRIMARY KEY,
card_id VARCHAR(50),
business_id VARCHAR(50),
timestamp TIMESTAMP,
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
4	16:08:17	SELECT * FROM transact.users_euro	0 row(s) returned	0.000 sec / 0.000 sec
5	16:08:48	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.265 sec

La tabla de clientes de EU tiene, como podemos apreciar, 3990 registros.

```
1 •  SELECT * FROM transact.users_euro;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	us
1000	Amkjrv	Qbulrxbp	+48-258-9936	amkjrv.qbulrxbp@example.com	May 17, 1970	Germany	Stuttgart	70173	215 Qbulrxbp St	2015-10-28	occ
1001	Nfvrlb	Oydaiwbg	+94-121-2522	nfvrlb.oydaiwbg@example.com	Mar 4, 1994	Germany	Cologne	50667	121 Oydaiwbg St	2014-05-22	pre
1002	Ijbfmd	Jbddzhvp	+70-120-3668	ijbfmd.jbddzhvp@example.com	Sep 27, 2001	Germany	Munich	80331	412 Jbddzhvp St	2013-02-11	stu
1003	Uyciig	Sfbdymzj	+58-123-6968	uyciig.sfbdymzj@example.com	Jan 20, 1981	Germany	Stuttgart	70173	735 Sfbdymzj St	2014-01-29	fan
1004	Yjqurq	Ojizvgqi	+77-944-2340	yjqurq.ojizvgqi@example.com	Jul 27, 1954	Germany	Munich	80331	685 Ojizvgqi St	2014-11-22	est
1005	Mnlqtu	Glofegwk	+54-801-2627	mnlqtu.glofegwk@example.com	Nov 15, 1962	Portugal	Funchal	9000-001	8 Glofegwk St	2015-01-13	occ
1007	Ehcrlp	Xahkzrlm	+71-862-6101	ehcrlp.xahkzrlm@example.com	Nov 8, 1971	Spain	Sevilla	41001	205 Xahkzrlm St	2013-02-18	you

Result Grid | Form Editor | Apply | Revert

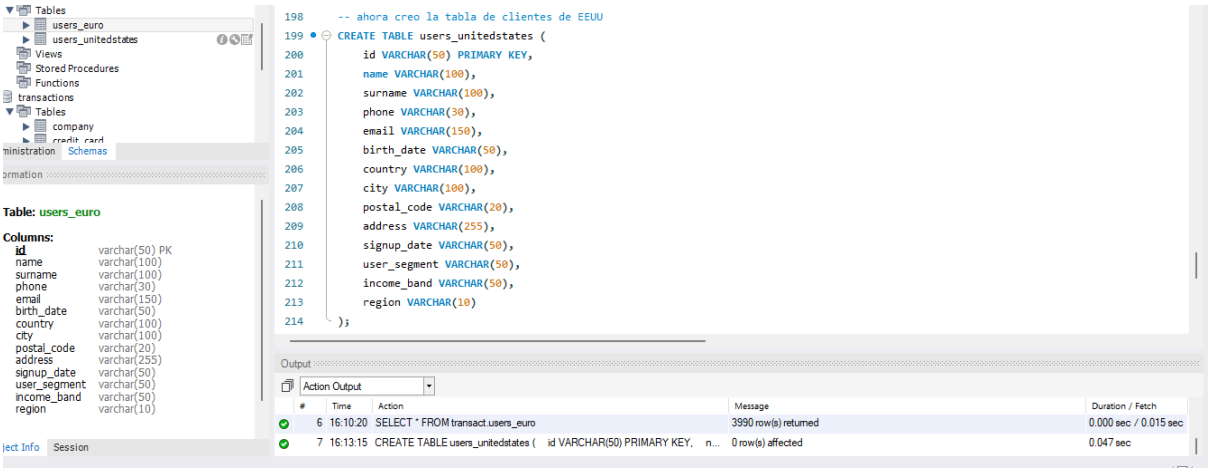
users_euro 4 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5	16:08:48	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.265 sec
6	16:10:20	SELECT * FROM transact.users_euro	3990 row(s) returned	0.000 sec / 0.015 sec

Ahora voy a crear la tabla de clientes de EE. UU., “users_unitedstates ” que tiene exactamente las mismas columnas con la diferencia de que en la región le aplicaremos “US”



Ahora cargamos los registros a esta nueva tabla. El sistema detectó 1010 registros.

```
217 •  LOAD DATA
218    INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__american_users.csv"
219    INTO TABLE users_euro
220    FIELDS TERMINATED BY ','
221    OPTIONALLY ENCLOSED BY '"'
222    LINES TERMINATED BY '\n'
223    IGNORE 1 LINES
224    (id, name, surname, phone, email, birth_date, country, city,
225     postal_code, address, signup_date, user_segment, income_band)
226    SET region = 'US'
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:18:29	CREATE TABLE IF NOT EXISTS transactions(id VARCHAR(50) PRIMARY KEY, ...	0 row(s) affected	0.046 sec
2	16:18:52	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\\...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec

El siguiente paso es crear una única tabla de usuarios donde añadimos los datos de ambas tablas diferenciando los clientes de EU y US.

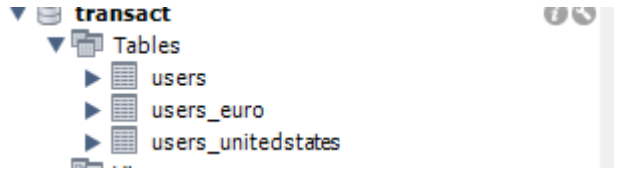
```
CREATE TABLE users (
    id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(30),
    email VARCHAR(150),
    birth_date VARCHAR(50),
    country VARCHAR(100),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address VARCHAR(255),
    signup_date VARCHAR(50),
    user_segment VARCHAR(50),
    income_band VARCHAR(50),
    region VARCHAR(10)
);

CREATE TABLE IF NOT EXISTS transactions(
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:24:13	CREATE TABLE users (id VARCHAR(50) PRIMARY KEY, name VARCHAR(...	0 row(s) affected	0.031 sec




```
SELECT * FROM transact.users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

email	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band	region
interdum.enim@protonmail.edu	Nov 17, 1985	United States	New York	10001	348-7818 Sagittis St.	2014-04-23	young_professional	high	US
fermentum@protonmail.com	Apr 30, 1984	United States	San Jose	95101	P.O. Box 773	2013-10-19	premium	high	US
risus.varius@google.ca	Sep 15, 1989	United States	San Jose	95101	Ap #644-8492 Sagittis St.	2015-04-20	frequent_shopper	very_high	US
amkjrv.qbulrxbp@example.com	May 17, 1970	Germany	Stuttgart	70173	215 Qbulrxbp St	2015-10-28	occasional	low	EU
nfvrlb.oydaiwbg@example.com	Mar 4, 1994	Germany	Cologne	50667	121 Oydaiwbg St	2014-05-22	premium	very_high	EU
ijbfmd.jbddzhvp@example.com	Sep 27, 2001	Germany	Munich	80331	412 Jbddzhvp St	2013-02-11	student	medium	EU
uyciig.sfbdymzj@example.com	Jan 20, 1981	Germany	Stuttgart	70173	735 Sfbdymzj St	2014-01-29	family	medium	EU

Apply

Como podemos apreciar, tenemos la tabla correctamente creada con el total de 5000 registros y la columna de región que funciona correctamente.

Último paso es la limpieza en los atributos de las columnas ya que la columna “birth_date y también la columna signup_date son de tipo VARCHAR y deberían ser de tipo DATE.
Ambas tienen diferentes formatos.

Primero modificaré el formato de la columna birth_date, pero como ya tengo las PK establecidas tengo que desactivar SAFE MODE (aunque sé que no es recomendable).

```
254     -- modifico el formato de la columna birth_date
255     -- pero como ya tengo las PK establecidas tengo que desactivar SAFE MODE (aunque sé que no es recomendable)
256 •   SET SQL_SAFE_UPDATES = 0;
257 •   UPDATE users
258     SET birth_date = DATE_FORMAT(
259         STR_TO_DATE(birth_date, '%b %d, %Y'),
260         '%Y-%m-%d'
261     )
262     WHERE birth_date LIKE '% % %';
263
264 •   INSERT INTO users (
265         id, name, surname, phone, email, birth_date,
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5	23:03:13	SELECT * FROM transact.users	5000 row(s) returned	0.000 sec / 0.047 sec
6	23:03:26	UPDATE users SET birth_date = DATE_FORMAT(STR_TO_DATE(birth_date, '%b %d, %...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.875 sec

Vuelvo a instalar SAFE MODE y procedo a actualizar el tipo de columna a DATE.

```
266     -- una vez comprobado que el formato ya se ve bien, procedo a actualizar el tipo de columna
267 •   ALTER TABLE users
268     MODIFY COLUMN birth_date DATE;
269
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	23:16:10	ALTER TABLE users MODIFY COLUMN birth_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.641 sec

Ahora modifico la columna signup_date de VARCHAR al DATE. Su formato ya estaba correcto.

```
266     -- modifico tipo de columna signup_date. EL formato estaba bien pero tipo era VARCHAR y lo cambio al DATE
267 •   ALTER TABLE users
268     MODIFY COLUMN signup_date DATE;
269
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	23:09:00	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
2	23:09:27	ALTER TABLE users MODIFY COLUMN signup_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	1.188 sec

Ahora elimino las tablas de users_euro y users_unitedstates. Todos sus registros ya estan en la tabla users.

```
267 • DROP TABLE users_unitedstates;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:45:40	DROP TABLE users_unitedstates	0 row(s) affected	0.047 sec

```
266 • DROP TABLE users_euro;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:45:13	DROP TABLE users_euro	0 row(s) affected	0.016 sec

A continuación, creo la tabla credit_cards. Tiene 5000 registros.

```
270 • CREATE TABLE IF NOT EXISTS credit_cards(
271       id VARCHAR(50) PRIMARY KEY,
272       user_id VARCHAR(15),
273       iban VARCHAR(34),
274       pan VARCHAR(19),
275       pin VARCHAR(4),
276       cvv VARCHAR(4),
277       track1 VARCHAR(150),
278       track2 VARCHAR(150),
279       expiring_date VARCHAR(10),
280       card_type VARCHAR (15),
281       card_renewal_flag BOOLEAN
282   );
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:45:40	DROP TABLE users_unitedstates	0 row(s) affected	0.047 sec
2	17:18:19	CREATE TABLE IF NOT EXISTS credit_cards(id VARCHAR(50) PRIMARY KEY, ...	0 row(s) affected	0.047 sec

Cargamos los registros.

```
285     -- cargamos los datos desde csv
286 •   LOAD DATA
287     INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__credit_cards.csv"
288     INTO TABLE credit_cards
289     FIELDS TERMINATED BY ','
290     LINES TERMINATED BY '\n'
291     IGNORE 1 LINES;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
4	17:22:42	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads...	Error Code: 1146. Table 'transact.cretid_cards' doesn't exist	0.000 sec
5	17:22:55	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.282 sec

Creo la tabla de dimensión, companies.

```sql
-- creamos la tabla "companies"
CREATE TABLE companies(
company_id VARCHAR(15) PRIMARY KEY,
company_name VARCHAR(150),
phone VARCHAR (50),
email VARCHAR (50),
country VARCHAR (50),
website VARCHAR(150),
merchant_category VARCHAR(50),
merchant_price_position VARCHAR(50)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:28:42	CREATE TABLE companies(company_id VARCHAR(15) PRIMARY KEY, compan...	0 row(s) affected	0.031 sec

Cargo los registros de la tabla companies.

```sql
-- cargamos los registros para la tabla de companies

LOAD DATA
INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__companies.csv"
INTO TABLE companies
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 LINES;
```

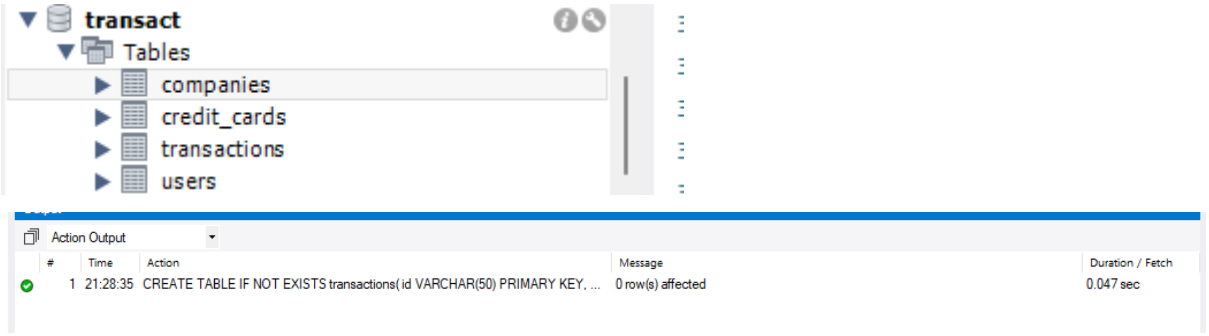
Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:30:40	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec

Ahora, procedo a crear la tabla de hechos, transactions.

```sql
CREATE TABLE IF NOT EXISTS transactions(
id VARCHAR(50) PRIMARY KEY,
card_id VARCHAR(50) REFERENCES credit_cards(id),
business_id VARCHAR(50) REFERENCES companies(company_id),
timestamp TIMESTAMP,
amount DECIMAL(10,2),
declined BOOLEAN,
products_ids TEXT,
user_id VARCHAR(50) REFERENCES users(id),
lat FLOAT,
longitude FLOAT,
discount_amount DECIMAL(10,2),
tax_amount DECIMAL(10,2),
shipping_amount DECIMAL(10,2),
channel VARCHAR(50),
campaign_id VARCHAR(50),
device_type VARCHAR(50),
is_international BOOLEAN,
decline_reason VARCHAR(255),
distance_km DECIMAL(8, 2)
  )
;
```



CREAMOS FK en la tabla de transactions para relacionarla con las demás tablas. Primero, uno users con transacions a través de su PK id y creo FK en user_id de la tabla transactions.

```
406  -- LA PARTE QUE FALTABA crear FOREIGN KEYS para relacionar las tablas
407  ALTER TABLE transactions
408  ADD CONSTRAINT fk_transaction_users
409  FOREIGN KEY (user_id)
410  REFERENCES users(id);
411
412
413
414
415
416  -- cargamos los registros para la tabla de transactions
417  LOAD DATA
418  INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__transactions.csv"
419  INTO TABLE transactions
420  FIELDS TERMINATED BY ';'
```

```
Output
Action Output
# Time Action Message Duration / Fetch
1 12:34:52 ALTER TABLE transactions ADD CONSTRAINT fk_transaction_users FOREIGN KEY (user_id) ... 99999 row(s) affected Records: 99999 Duplicates: 0 Warnings: 0 4.828 sec
```

Ahora, vinculamos credit_cards con transactions.

```
413  ALTER TABLE transactions
414  ADD CONSTRAINT fk_transactions_credit_cards
415  FOREIGN KEY (card_id)
416  REFERENCES credit_cards(id);
417
418
419
420  -- cargamos los registros para la tabla de transactions
```

```
Output
Action Output
# Time Action Message Duration / Fetch
1 12:40:24 ALTER TABLE transactions ADD CONSTRAINT fk_transactions_credit_cards FOREIGN KEY (... 99999 row(s) affected Records: 99999 Duplicates: 0 Warnings: 0 5.281 sec
```

Ahora, vinculamos companies con transactions.

```
418  -- vinculo companies con transactions
419  ALTER TABLE transactions
420  ADD CONSTRAINT fk_transactions_companies
421  FOREIGN KEY (business_id)
422  REFERENCES companies(company_id);
```

```
Output
Action Output
# Time Action Message Duration / Fetch
1 12:42:50 ALTER TABLE transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (b... 99999 row(s) affected Records: 99999 Duplicates: 0 Warnings: 0 3.812 sec
```

La tabla transactions y products están vinculadas a través de una tabla intermedia.

Procedemos a cargar los registros a la tabla y hay 10000 registros.

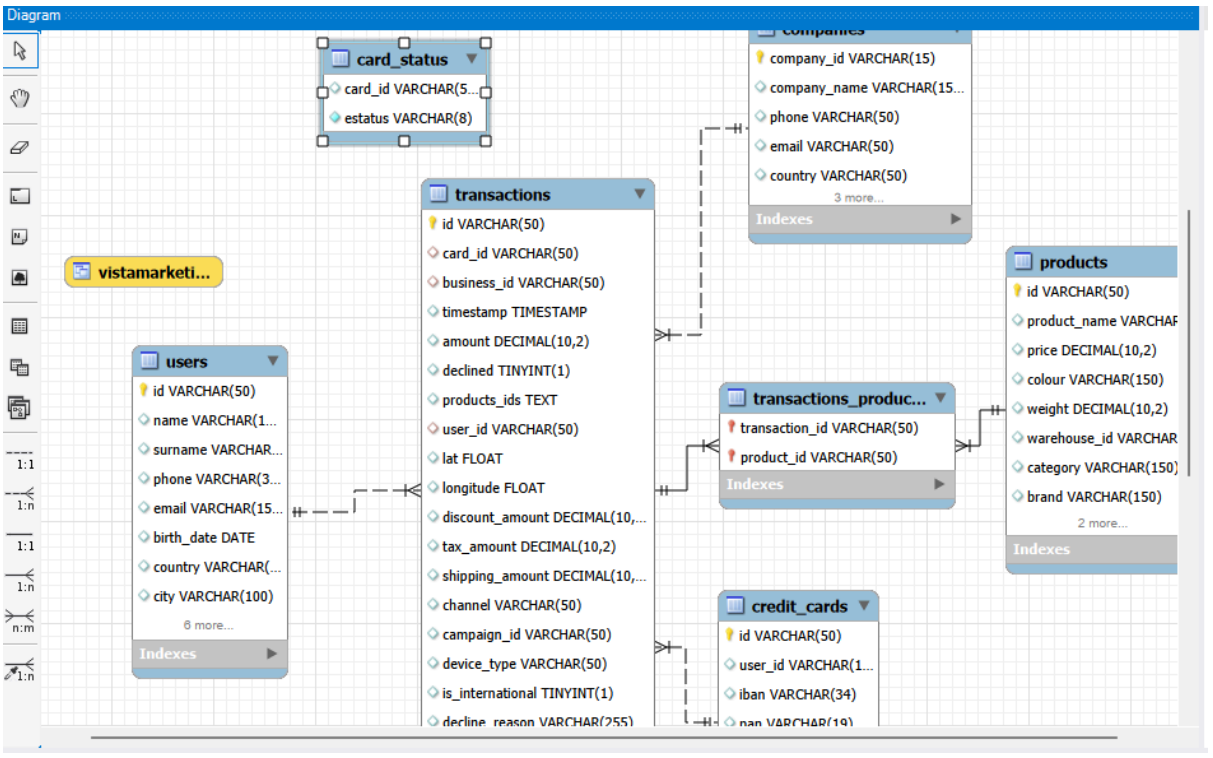
```
337     -- cargamos los registros para la tabla de transactions
338     LOAD DATA
339     INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__transactions.csv"
340     INTO TABLE transactions
341     FIELDS TERMINATED BY ';'
342     OPTIONALLY ENCLOSED BY '"'
343     LINES TERMINATED BY '\n'
344     IGNORE 1 LINES;
345
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	23:10:10	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	3.359 sec
11	13:36:26	CREATE TABLE IF NOT EXISTS transactions(id VARCHAR(50) PRIMARY KE...	0 row(s) affected	0.047 sec

NUEVO DIAGRAMA CON FK INTRODUCIDAS:



EJERCICIO 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```sql
-- Ejercicio 9
-- Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.
SELECT * FROM users
WHERE id IN (
    SELECT user_id FROM transactions
    GROUP BY user_id
    HAVING COUNT(id) > 80);

-- EJERCICIO 10
```

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	user_segment
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.	2013-08-11	frequent_shopper
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany	Stuttgart	70173	82 Hwcru Street	2013-12-11	occasional
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa	16100	53 Astuw Street	2013-05-29	frequent_shopper
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	Aug 28, 1962	Poland	Gdansk	80-001	52 Xgvfridxs Street	2014-08-30	established

#	Time	Action	Message	Duration / Fetch
1	00:09:53	SELECT * FROM users WHERE id IN (SELECT user_id FROM transactions GROUP BY us...	4 row(s) returned	0.110 sec / 0.000 sec

EJERCICIO 10

Muestra la media de la cantidad de transacciones (amount) por IBAN de las tarjetas de crédito en la compañía Donec Ltd., Utiliza por lo menos 2 tablas.

```sql
SELECT c.iban, ROUND(AVG(t.amount),2) AS media FROM transactions t
JOIN credit_cards c
ON t.card_id=c.id
JOIN companies comp
ON comp.company_id = t.business_id
WHERE comp.company_name ="Donec Ltd"
GROUP BY c.iban;
```

iban	media
CA90135577358677162610899O	361.01
US729212642816955651428791	142.56
US899450803230933276819251	246.90
US581097432668325541142297	154.01
CA441226267105857473570574	250.51

#	Time	Action	Message	Duration / Fetch
1	21:58:09	SELECT c.iban, ROUND(AVG(t.amount),2) AS media FROM transactions t JOIN credit_cards ...	371 row(s) returned	0.438 sec / 0.000 sec

NIVEL 2

EJERCICIO 1

Identifica los cinco días en que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

Le aplico la función DATE a la columna timestamp ya que solo nos piden por fecha (sin hora exacta).

Corrijo para que declined <>1 (que nos deje solo las transacciones aceptadas) y quito lo del descuento.

```
459 •  SELECT DATE(timestamp) AS fecha, ROUND(SUM(amount),2) AS total_ventas
460    FROM transactions
461    WHERE declined <>1
462    GROUP BY DATE(timestamp)
463   ORDER BY total_ventas DESC
464    LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fecha	total_ventas
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

Result 14 | Read Only

Output

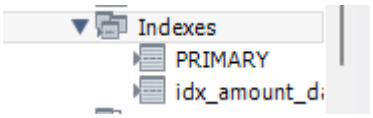
Action Output

#	Time	Action	Message	Duration / Fetch
1	13:16:34	SELECT DATE(timestamp) AS fecha, ROUND(SUM(amount),2) AS total_ventas FROM trans...	5 row(s) returned	0.109 sec / 0.000 sec

EJERCICIO 2

Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor comprendido entre 350 y 400 euros y en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.

En este caso, vi oportuno crear un index B-tree (secundario) porque está claro que se va a recurrir mucho a la columna de amount y de la fecha en la tabla de transacciones




```
380 •  CREATE INDEX idx_amount_date
381    ON transactions(amount, timestamp);
```

```
412 •  SELECT c.company_name, c.phone, c.country,t.timestamp AS date, ROUND(t.amount,2) AS Amount
413    FROM companies c
414    JOIN transactions t
415       ON c.company_id= t.business_id
416    WHERE ROUND(t.amount,2) BETWEEN 350 AND 400
417    AND DATE(timestamp) IN
418    ("2015-04-29", "2018-07-20", "2024-03-13")
419    ORDER BY ROUND(t.amount,2) DESC;
420
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	date	Amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29 06:18:25	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13 01:23:09	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29 21:24:49	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20 14:57:32	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13 13:20:45	352.21

Result 81 | Read Only

Output

Action Output

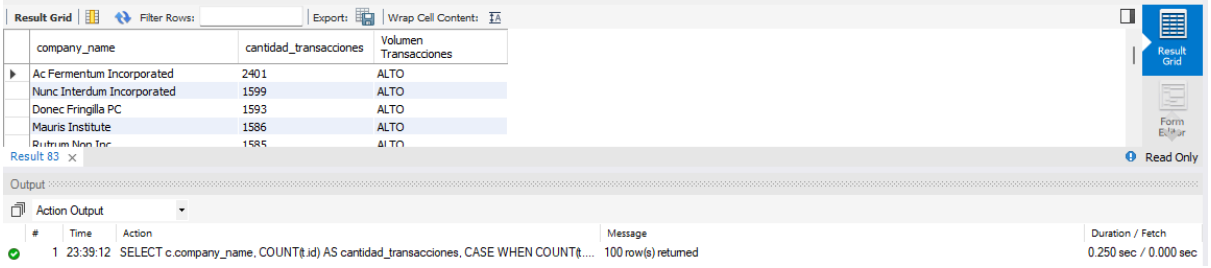
#	Time	Action	Message	Duration / Fetch
1	23:37:15	SELECT c.company_name, c.phone, c.country,t.timestamp AS date, ROUND(t.amount,2) AS ...	6 row(s) returned	0.063 sec / 0.000 sec

EJERCICIO 3

Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información

sobre la cantidad de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen 400 transacciones o igual o si tienen menos de 400.

```sql
SELECT c.company_name, COUNT(t.id) AS cantidad_transacciones,
    CASE
        WHEN COUNT(t.id) >= 400 THEN "ALTO"
        ELSE "BAJO"
    END AS 'Volumen Transacciones'
FROM companies c
JOIN transactions t
    ON c.company_id = t.business_id
GROUP BY c.company_name
ORDER BY cantidad_transacciones DESC;
```



company_name	cantidad_transacciones	Volumen Transacciones
Ac Fermentum Incorporated	2401	ALTO
Nunc Interdum Incorporated	1599	ALTO
Donec Fringilla PC	1593	ALTO
Mauris Institute	1586	ALTO
Rutrum Non Inc	1585	ALTO

Result 83 — Read Only

Output

Action Output

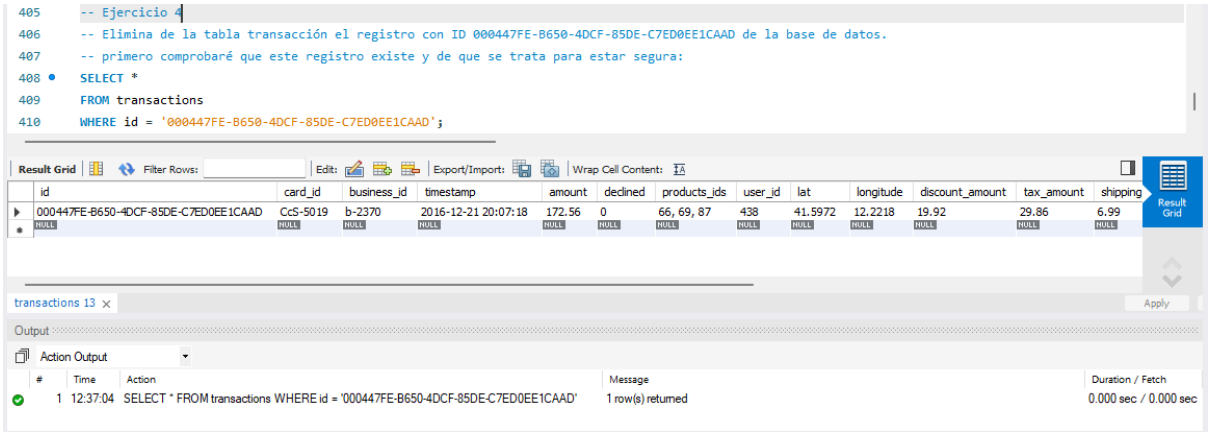
#	Time	Action	Message	Duration / Fetch
1	23:39:12	SELECT c.company_name, COUNT(t.id) AS cantidad_transacciones, CASE WHEN COUNT(t....	100 row(s) returned	0.250 sec / 0.000 sec

EJERCICIO 4

Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.

Primero comprobaré que este registro existe y de qué se trata para estar segura:

```sql
-- Ejercicio 4
-- Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
-- primero comprobaré que este registro existe y de que se trata para estar segura:
SELECT *
FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```



id	card_id	business_id	timestamp	amount	declined	products_ids	user_id	lat	longitude	discount_amount	tax_amount	shipping
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.5972	12.2218	19.92	29.86	6.99

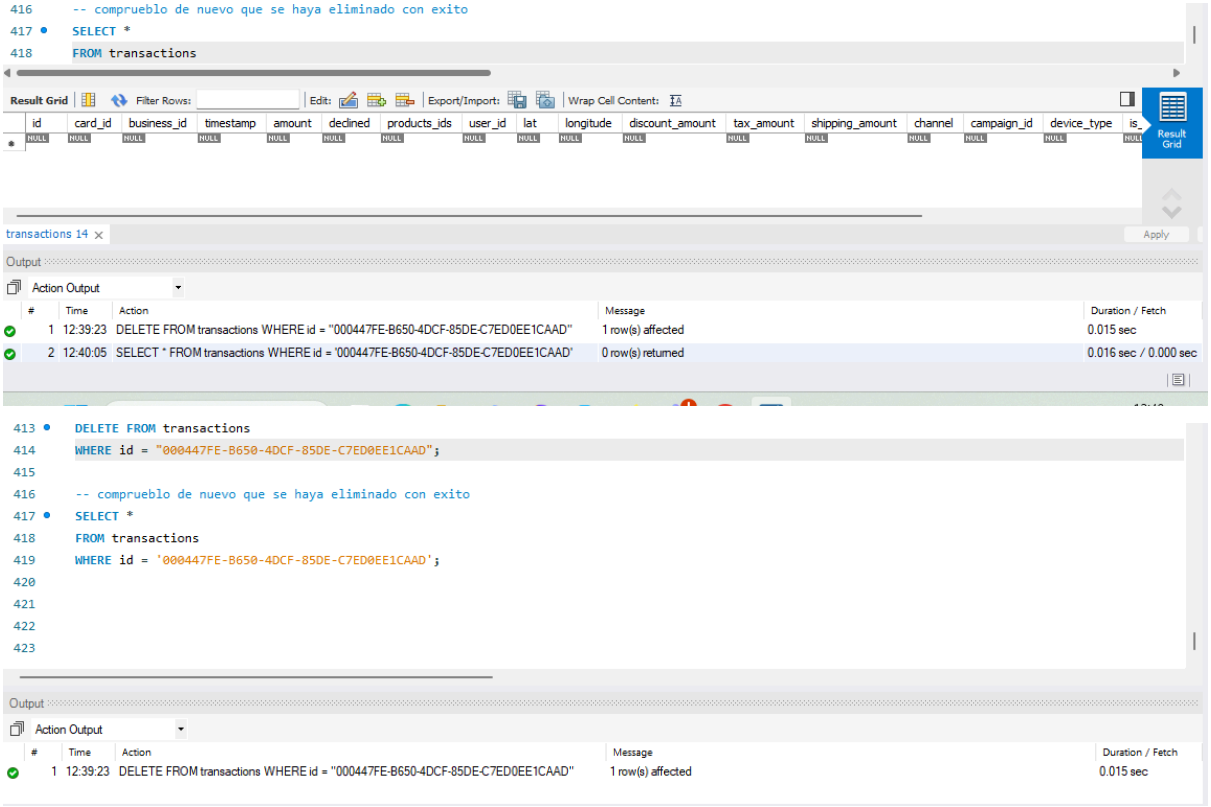
transactions 13 — Apply

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:37:04	SELECT * FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned	0.000 sec / 0.000 sec

Ahora procedo a borrarlo y de nuevo busco este registro. Me tiene que dar 0 registros si la acción fue exitosa.



.

EJERCICIO 5

La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones. Será necesario que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizada por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra. Añado DROP VIEW IF EXISTS y en SELECT company_id para evitar problemas si hay dos empresas con el mismo nombre p.e.

```sql
SELECT c.company_id, c.company_name, c.phone, c.country, ROUND(AVG(t.amount),2) AS media_compra
FROM companies AS c
JOIN transactions AS t
    ON c.company_id = t.business_id
GROUP BY c.company_name, c.phone, c.country;

-- reviso que funciona
SELECT * FROM  `VistaMarketing`;

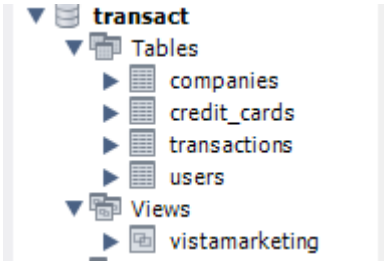
-- ordeno la vista del mayor a menor
SELECT * FROM  `VistaMarketing`
ORDER BY media_compra DESC;


-- NIVEL 3
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	13:13:36	CREATE VIEW `VistaMarketing` AS SELECT c.company_id, c.company_name, c.phone, c.cou...	0 row(s) affected	0.000 sec



Finalmente, ordeno la vista de mayor a menor

```sql
-- ordeno la vista del mayor a menor
SELECT * FROM  `VistaMarketing`
ORDER BY media_compra DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	media_compra
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.77
Ut Semper Foundation	+10 283 500 6449	Sweden	307.37
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	306.27
Dolor Vitae Limited	+6 421 567 9195	France	305.95
Nunc In Foundation	+13 267 741 2619	Italy	305.40

VistaMarketing 89

Output

Action Output

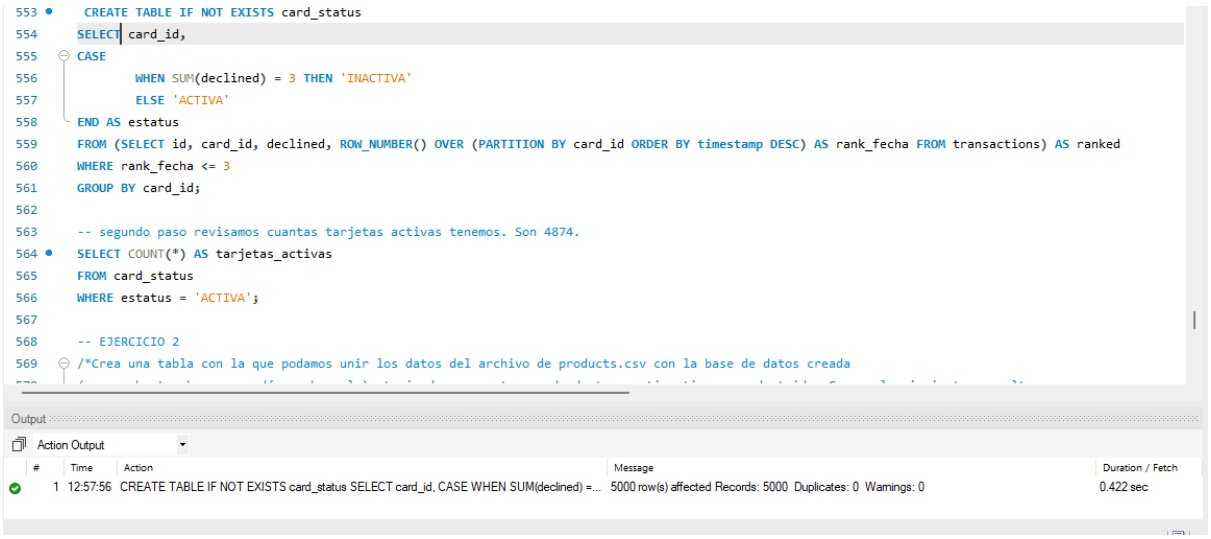
#	Time	Action	Message	Duration / Fetch
5	11:01:39	SELECT * FROM `VistaMarketing`	100 row(s) returned	1.641 sec / 0.000 sec
6	11:04:10	SELECT * FROM `VistaMarketing` ORDER BY media_compra DESC	100 row(s) returned	0.844 sec / 0.000 sec

NIVEL 3

EJERCICIO 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si por lo menos una no es rechazada entonces es activo. Partiendo de esta tabla responde: ¿Cuántas tarjetas están activas?

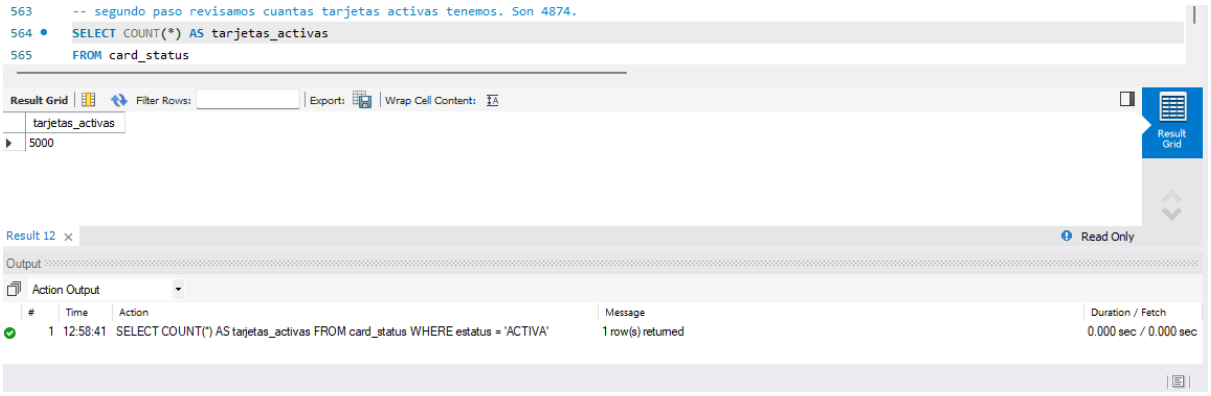
Primer paso es crear la nueva tabla “card_status”



```
553    CREATE TABLE IF NOT EXISTS card_status
554    SELECT card_id,
555    CASE
556            WHEN SUM(declined) = 3 THEN 'INACTIVA'
557            ELSE 'ACTIVA'
558    END AS estatus
559    FROM (SELECT id, card_id, declined, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS rank_fecha FROM transactions) AS ranked
560    WHERE rank_fecha <= 3
561    GROUP BY card_id;
562
563    -- segundo paso revisamos cuantas tarjetas activas tenemos. Son 4874.
564    SELECT COUNT(*) AS tarjetas_activas
565    FROM card_status
566    WHERE estatus = 'ACTIVA';
567
568    -- EJERCICIO 2
569    /*Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada
```

```
Output
Action Output
#  Time      Action                                                                                                  Message                                                                  Duration / Fetch
1  12:57:56  CREATE TABLE IF NOT EXISTS card_status SELECT card_id, CASE WHEN SUM(declined) =...  5000 row(s) affected Records: 5000  Duplicates: 0  Warnings: 0  0.422 sec
```

Segundo paso, revisamos cuántas tarjetas activas tenemos. Son 5000.



```
563    -- segundo paso revisamos cuantas tarjetas activas tenemos. Son 4874.
564    SELECT COUNT(*) AS tarjetas_activas
565    FROM card_status
```

```
Result Grid | Filter Rows: | Export: | Wrap Cell Content:
tarjetas_activas
5000
Result Grid
Result 12 ×
Read Only
Output
Action Output
#  Time      Action                                                                                  Message                Duration / Fetch
1  12:58:41  SELECT COUNT(*) AS tarjetas_activas FROM card_status WHERE estatus = 'ACTIVA'  1 row(s) returned  0.000 sec / 0.000 sec
```

EJERCICIO 2

Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada (ya que hasta ahora no podíamos hacerlo), teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta: Necesitamos conocer el número de veces que se ha vendido cada producto.

Primero creamos la tabla products y llenaremos los datos. Despues crearemos la tabla intermedia transactions_products que une las dos

```sql
CREATE TABLE products(
    id VARCHAR(50) PRIMARY KEY,
    product_name VARCHAR(150),
    price DECIMAL(10, 2),
    colour VARCHAR(150),
    weight DECIMAL(10, 2),
    warehouse_id VARCHAR(15),
    category VARCHAR(150),
    brand VARCHAR(150),
    cost DECIMAL(10, 2),
    launch_date DATE
    )
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:00:07	CREATE TABLE products(id VARCHAR(50) PRIMARY KEY, product_name VARCHAR(150), p...	0 row(s) affected	0.047 sec

```sql
-- ahora llenamos la tabla products con los registros segun el documento csv. Tuve que eliminar el simbolo de dolar de los registros.
LOAD DATA
INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__products.csv"
INTO TABLE products
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(
    id,product_name, @price, colour, weight, warehouse_id,category,brand,@cost,launch_date
)
SET
    price = REPLACE(@price, '$', ''),
    cost = REPLACE(@cost, '$', '');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
6	12:12:44	SELECT * FROM transact.products	100 row(s) returned	0.000 sec / 0.000 sec

```sql
SELECT * FROM transact.products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	product_name	price	colour	weight	warehouse_id	category	brand	cost	launch_date
1	Direwolf Stannis	161.11	#7c7c7c	1.00	WH-4	Sports	Runway Pro	63.84	2014-11-01
10	Karstark Dorne	119.52	#f4f4f4	2.40	WH-5	Food	FreshBox	89.07	2013-10-04
100	south duel	40.43	#6d6d6d	3.00	WH-95	Electronics	PixelForge	24.95	2013-01-05
11	Karstark Dorne	49.70	#141414	2.70	WH-6	Electronics	Nexa	34.45	2014-09-01
12	duel Direwolf	181.60	#a8a8a8	2.10	WH-7	Gaming	Questbyte	99.46	2013-10-11

Ahora creamos la tabla intermedia entre transactions y products ya que la columna de producs_id en transactions conlleva varios productos

```sql
CREATE TABLE IF NOT EXISTS transactions_products(
    transaction_id VARCHAR(50),
    product_id VARCHAR(50),
    PRIMARY KEY (transaction_id, product_id),-- en una tabla intermedia que PK tiene que ser compuestos de los dos
    FOREIGN KEY (transaction_id) REFERENCES transactions(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
    );
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:48:10	CREATE TABLE IF NOT EXISTS transactions_products(transaction_id VARCHAR(50), prod...	0 row(s) affected	0.079 sec

Para poder llenar correctamente la tabla tenemos que convertir JSON de la columna products_ids en tabla donde cada producto de la transacciones este en una fila.

```
503 •  INSERT INTO transactions_products (transaction_id, product_id)
504    SELECT t.id, jt.product_id
505    FROM transactions t
506    JOIN JSON_TABLE(
507        CONCAT('[', t.products_ids, ']'),
508        '$[*]' COLUMNS (product_id INT PATH '$')
509    ) jt;
510
511
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
27	14:50:21	SELECT product_id, ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY prod...	253388 row(s) returned	0.266 sec / 0.172 sec

El último paso es hacer la consulta sobre el número de veces que se ha vendido cada producto. Fitramos por solo transacciones aceptadas.

```
622    -- Calculamos el número de veces que se ha vendido cada producto
623 •  SELECT p.id AS id, p.product_name AS product_name, COUNT(tp.product_id) AS times_sold
624    FROM products p
625    LEFT JOIN transactions_products tp
626    ON p.id=tp.product_id
627    LEFT JOIN transactions t
628    ON t.id=tp.transaction_id
629    WHERE  t.declined = 0
630    GROUP BY p.id,p.product_name
631    ORDER BY times_sold DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_name	times_sold
52	riverlands the duel	2635
29	Tully maester Tarly	2607
16	the duel warden	2583
48	rock Renly in	2580
21	duel Direwolf	2579

Result 13 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	13:08:34	SELECT p.id AS id, p.product_name AS product_name, COUNT(tp.product_id) AS times_sold ...	100 row(s) returned	2.453 sec / 0.000 sec